DATABASE ARCHITECTURE DOCUMENT

RiskHub MongoDB

NoSQL Schema Design & Indexing Strategy

4 Core Collections — Optimized for Time-Series & Real-Time Quant Workloads

Document Type	Database Architecture & Schema Design
Version	1.0 — MVP
Audience	Backend Engineers, Database Architects, DevOps
Collections	users | trade_history | risk_metrics | alerts_log
Database	MongoDB 7.x (Atlas or self-hosted replica set)
Prepared By	Senior Database Architect / Backend Tech Lead
Date	April 2025

0. Architecture Overview & Design Principles

This document specifies the complete MongoDB NoSQL schema for the RiskHub platform. Four collections form the core data layer, each designed for a specific access pattern driven by the Behavioral Quant Engine, the real-time alert pipeline, and the SBT (Soulbound Token) Oracle workflow.

Core Design Philosophy

Pattern-First Design: Every field exists because a specific query or computation requires it. No speculative fields. Every index has a named query pattern it serves. Denormalization is deliberate — we embed data that is always read together to eliminate JOIN-equivalent $lookup operations on hot paths.

0.1 Collection Responsibility Map

Collection	Primary Role	Write Pattern	Read Pattern	Est. Volume
users	Identity, credentials, SBT status	Low (user actions)	Point lookup by _id / email	< 100K docs

`© 2025 RiskHub. Confidential.

Collection	Primary Role	Write Pattern	Read Pattern	Est. Volume
trade_history	Raw trade ledger from all exchanges	High (60s polling)	Time-range scan + sort	50M+ docs @ scale
risk_metrics	Quant Engine output snapshots	Medium (per sync cycle)	Latest-first + range for charts	10M+ docs @ scale
alerts_log	Behavioral alert events	Event-driven (burst)	Recent unread + paginated	5M+ docs @ scale

0.2 Global Design Decisions

- **ObjectId for _id:** MongoDB default ObjectId is used throughout. It encodes a timestamp, enabling time-based document ordering as a free secondary property.
- **Decimal128 for financial values:** All monetary amounts, percentages, and ratios use NumberDecimal (Decimal128) to avoid IEEE 754 floating-point rounding errors in PnL calculations.
- **ISODate for all timestamps:** UTC-normalized ISODate is used for every timestamp. Application layer converts to user timezone for display only.
- **AES-256-GCM encryption at application layer:** Exchange API keys are encrypted by the FastAPI backend before any write to MongoDB. The database never holds plaintext credentials. The envelope format is enc::iv::tag::ciphertext.
- **Compound indexes lead with user_id:** All multi-field indexes place user_id as the first component. This is the primary equality filter for all queries, maximizing index selectivity.
- **TTL indexes for automatic data hygiene:** Ephemeral data (SIWE nonces, old alert records, stale metrics) is automatically purged by MongoDB TTL indexes, eliminating scheduled cleanup jobs.
- **Schema versioning:** Every document includes schema_version: 1. Future migrations can target specific versions in bulk update scripts without full collection scans.

1. Collection: users

1.1 Purpose

The users collection is the identity root of the entire platform. It stores authentication credentials, linked Web3 wallet addresses, encrypted exchange API key pairs (the most security-sensitive data in the system), and the user's current Soulbound Token minting state. All other collections reference users._id as their primary partition key.

1.2 Schema Document

```
// users — Document Schema (MongoDB JSON)
```

```
{
  _id: ObjectId("64f1a2b3c4d5e6f7a8b9c0d1"),

  // ── Identity ──────────────────────────────────────────────
  email: "trader@example.com",          // String, unique, indexed
  password_hash: "bcrypt_v2$...",       // String, bcrypt hash (NOT stored in
plaintext)
  username: "crypto_minh",              // String, unique, sparse index
  created_at: ISODate("2024-01-15T08:00:00Z"),  // Date
  updated_at: ISODate("2024-03-10T14:22:00Z"),  // Date
  last_login_at: ISODate("2024-03-10T14:22:00Z"), // Date
  is_active: true,                      // Boolean
  email_verified: true,                 // Boolean
  email_verified_at: ISODate("2024-01-15T08:05:00Z"), // Date

  // ── Web3 Identity ─────────────────────────────────────────
  wallet: {
    address: "0xAbCd...EfGh",           // String, lowercase, sparse unique index
    chain_id: 56,                       // Number (56=BNB Chain, 1=Ethereum)
    linked_at: ISODate("2024-01-20T10:00:00Z"), // Date
    siwe_nonce: "a3f9b2c1",             // String, one-time nonce for SIWE flow
    siwe_nonce_expires_at: ISODate("2024-03-10T14:30:00Z"), // Date, TTL candidate
  },

  // ── Soulbound Token (SBT) ─────────────────────────────────
  sbt: {
    status: "minted",                   // Enum: "none" | "pending" | "minted" |
"error"
    token_id: "1042",                   // String (on-chain token ID)
    contract_address: "0x1234...ABCD",  // String (deployed contract)
    network: "bnb_testnet",             // Enum:
"bnb_testnet"|"sepolia"|"bnb_mainnet"|"mainnet"
    tx_hash: "0xdeadbeef...",           // String, transaction hash
    minted_at: ISODate("2024-02-01T09:30:00Z"), // Date
    last_updated_at: ISODate("2024-03-05T11:00:00Z"), // Date
    metrics_snapshot_id: ObjectId("64f2..."), // Ref -> risk_metrics._id at mint
time
    oracle_signature: "0xabc123...",    // String, ECDSA signature from backend
oracle
  },

  // ── Exchange API Keys (ENCRYPTED at application layer) ────
  // WARNING: api_key and api_secret MUST be encrypted with AES-256-GCM
  // before write. Decryption occurs in-memory only. NEVER log these fields.
  exchange_keys: [
    {
      exchange_id: "binance",           // String, CCXT exchange identifier
      label: "Binance Main",            // String, user-defined label
      api_key_encrypted: "enc::iv::tag::ciphertext",  // [AES-256-GCM ENCRYPTED]
      api_secret_encrypted: "enc::iv::tag::ciphertext", // [AES-256-GCM ENCRYPTED]
      passphrase_encrypted: null,       // [AES-256-GCM ENCRYPTED] (OKX/Gate.io
only)
      permissions_verified: ["read"],   // Array<String>, validated on submission
      is_active: true,                  // Boolean
      last_sync_at: ISODate("2024-03-10T14:00:00Z"), // Date
      last_sync_status: "ok",           // Enum: "ok" | "error" | "rate_limited"
      last_sync_error: null,            // String | null, last error message
      added_at: ISODate("2024-01-16T09:00:00Z"), // Date
    },
    {
      exchange_id: "okx",
      label: "OKX Futures",
      api_key_encrypted: "enc::iv2::tag2::cipher2",    // [AES-256-GCM ENCRYPTED]
      api_secret_encrypted: "enc::iv2::tag2::cipher2", // [AES-256-GCM ENCRYPTED]
      passphrase_encrypted: "enc::iv3::tag3::cipher3", // [AES-256-GCM ENCRYPTED]
```

```
      permissions_verified: ["read"],
      is_active: true,
      last_sync_at: ISODate("2024-03-10T13:55:00Z"),
      last_sync_status: "ok",
      last_sync_error: null,
      added_at: ISODate("2024-01-18T11:30:00Z"),
    }
  ],

  // ── Auth Tokens ──────────────────────────────────────────
  refresh_token_hash: "sha256_hash_of_refresh_token", // String, hashed
  refresh_token_expires_at: ISODate("2024-03-17T14:22:00Z"), // Date

  // ── Preferences ──────────────────────────────────────────
  preferences: {
    alerts_enabled: true,               // Boolean
    alert_channels: ["websocket"],      // Array<String>: "websocket"|"email"
    timezone: "Asia/Ho_Chi_Minh",       // String
    currency_display: "USD",            // String
  }
}
```

1.3 Index Definitions

```
// users — createIndex() Statements
// ── Users Collection Indexes ──────────────────────────────
db.users.createIndex({ email: 1 }, { unique: true, name: "idx_email_unique" });

db.users.createIndex(
  { "wallet.address": 1 },
  { unique: true, sparse: true, name: "idx_wallet_address_unique" }
  // sparse: true because not all users link a wallet at registration
);

db.users.createIndex(
  { "exchange_keys.exchange_id": 1 },
  { name: "idx_exchange_keys_exchange_id" }
  // Enables: find users connected to a specific exchange (ops/monitoring)
);

db.users.createIndex(
  { "sbt.status": 1, "sbt.token_id": 1 },
  { sparse: true, name: "idx_sbt_status_tokenid" }
  // Enables: dashboard queries for pending SBT jobs; token ID lookups
);

db.users.createIndex(
  { "wallet.siwe_nonce_expires_at": 1 },
  { expireAfterSeconds: 0, name: "idx_siwe_nonce_ttl" }
  // TTL index: MongoDB auto-deletes expired SIWE nonces — no cron required
);
```

1.4 Design Rationale

Embedded Exchange Keys Array

Exchange API keys are stored as an embedded array inside the user document (exchange_keys[]) rather than in a separate collection. This is a deliberate denormalization choice: the frontend dashboard always needs both user profile data and exchange connection status in a single read. Embedding eliminates a $lookup. The array is bounded in size (users typically connect 2–5 exchanges), so it never causes document growth issues.

AES-256-GCM Encryption Envelope

> **SECURITY CRITICAL — Application-Layer Encryption**
> Fields api_key_encrypted, api_secret_encrypted, and passphrase_encrypted are NEVER stored in plaintext. The FastAPI backend encrypts them using AES-256-GCM before any database write. Format: enc::<base64_iv>::<base64_tag>::<base64_ciphertext>. The master encryption key is stored only in environment secrets (AWS Secrets Manager / HashiCorp Vault). If the MongoDB database is compromised, encrypted key blobs are useless without the master key. Raw API keys are zeroed from memory immediately after encryption.

SBT Subdocument — Embedded vs Reference

The SBT status object is embedded in the user document rather than in a separate collection because: (1) there is exactly one SBT per user (1:1 relationship); (2) the dashboard always reads SBT status alongside user profile data; (3) atomic updates to both user data and SBT status happen in a single document operation, which is trivially atomic in MongoDB without transactions.

Sparse Index on wallet.address

The unique index on wallet.address uses sparse: true because wallet connection is optional — users can use RiskHub with email/password only. A non-sparse unique index would prevent more than one user from having a null wallet address, which would break email-only registration. A sparse index ignores null/absent fields entirely.

2. Collection: trade_history

2.1 Purpose

The trade_history collection is the largest and most write-intensive collection in RiskHub. It stores every closed trade position fetched from every connected exchange for every user. It is the primary input to the Behavioral Quant Engine — all behavioral rules (Revenge Trading, Overtrading, Leverage Escalation), the Discipline Score, Win Rate, and Max Drawdown are computed by querying this collection. The schema is optimized for high-volume time-range scans sorted by recency.

2.2 Schema Document

```
// trade_history — Document Schema (MongoDB JSON)
{
  _id: ObjectId("65a3b4c5d6e7f8g9h0i1j2k3"),

  // ── Partition Keys (Critical for Index Performance) ───────
  user_id: ObjectId("64f1a2b3c4d5e6f7a8b9c0d1"), // Ref -> users._id, INDEXED
  exchange_id: "binance",                // String, CCXT id, INDEXED
  account_type: "futures",              // Enum: "spot" | "futures"
```

```
  // ── Exchange-native identifiers ──────────────────────────
  exchange_trade_id: "8317492834",     // String, unique per exchange (for dedup)
  exchange_order_id: "4829374652",     // String, parent order ID

  // ── Instrument & Position ────────────────────────────────
  symbol: "BTCUSDT",                   // String, CCXT unified symbol, INDEXED
  base_asset: "BTC",                   // String (derived, stored for fast filter)
  quote_asset: "USDT",                 // String (derived)
  side: "long",                        // Enum: "long" | "short" (normalized from
buy/sell)
  position_side: "long",               // Enum: "long" | "short" | "both" (hedge
mode)

  // ── Execution Metrics (Core Quant Engine Inputs) ─────────
  leverage: 20,                        // Number (integer), 1 = spot/no leverage
  entry_price: NumberDecimal("43250.00"),  // Decimal128, average fill price
  exit_price: NumberDecimal("41800.00"),   // Decimal128, average close price
  quantity: NumberDecimal("0.05"),     // Decimal128, base asset quantity
  notional_value_usd: NumberDecimal("2162.50"), // Decimal128, quantity *
exit_price
  margin_used_usd: NumberDecimal("108.13"),      // Decimal128, notional /
leverage

  // ── PnL ──────────────────────────────────────────────────
  realized_pnl_usd: NumberDecimal("-72.50"),  // Decimal128, net PnL after fees
  realized_pnl_pct: NumberDecimal("-3.35"),   // Decimal128, PnL as % of margin
  gross_pnl_usd: NumberDecimal("-72.50"),     // Decimal128, before fees
  fee_usd: NumberDecimal("2.16"),             // Decimal128, total commission paid
  funding_fee_usd: NumberDecimal("-0.84"),    // Decimal128, futures funding
payments

  // ── Outcome (Derived, Stored for Query Performance) ──────
  is_win: false,                       // Boolean, realized_pnl_usd > 0
  pnl_category: "loss",               // Enum: "win" | "loss" | "breakeven"

  // ── Timestamps (CRITICAL for time-series queries) ────────
  opened_at: ISODate("2024-03-10T08:00:00Z"),   // Date, position open time,
INDEXED
  closed_at: ISODate("2024-03-10T09:15:00Z"),   // Date, position close time,
INDEXED
  duration_seconds: 4500,             // Number, derived: closed_at - opened_at

  // ── Raw Exchange Response (for auditability) ─────────────
  raw_exchange_data: {                 // Object, original CCXT response payload
    ccxt_trade_id: "8317492834",
    ccxt_symbol: "BTC/USDT:USDT",
    original_side: "sell",            // exchange-native side before normalization
    info: {}                          // raw exchange 'info' object (exchange-
specific)
  },

  // ── Metadata ─────────────────────────────────────────────
  synced_at: ISODate("2024-03-10T14:05:00Z"),   // Date, when RiskHub ingested
this record
  schema_version: 1                   // Number, for future migrations
}
```

2.3 Index Definitions & Query Examples

```
// trade_history — createIndex() Statements + Query Examples
// ── Trade History Collection Indexes ──────────────────────
// PRIMARY WORKHORSE: Supports all Quant Engine queries
// Filter by user + exchange, sorted by recency. Covers:
//   - 30-day trade fetch for Discipline Score
//   - Revenge Trading rule (recent trades post-loss)
//   - Per-exchange portfolio history
db.trade_history.createIndex(
  { user_id: 1, exchange_id: 1, closed_at: -1 },
  { name: "idx_user_exchange_closedat_desc" }
);

// DEDUPLICATION: Prevent duplicate ingestion from exchange polling
db.trade_history.createIndex(
  { user_id: 1, exchange_id: 1, exchange_trade_id: 1 },
  { unique: true, name: "idx_dedup_trade" }
);

// SYMBOL FILTER: Contagion graph — trades by asset symbol
db.trade_history.createIndex(
  { user_id: 1, symbol: 1, closed_at: -1 },
  { name: "idx_user_symbol_closedat" }
);

// OPENED_AT: Revenge Trading rule — query trades opened within N minutes
db.trade_history.createIndex(
  { user_id: 1, opened_at: -1 },
  { name: "idx_user_openedat_desc" }
);

// WIN/LOSS STREAK: Fast consecutive-loss detection
db.trade_history.createIndex(
  { user_id: 1, is_win: 1, closed_at: -1 },
  { name: "idx_user_iswin_closedat" }
);

// EXAMPLE QUERIES using indexes above:
// 1. Quant Engine: last 30 days of trades for a user
db.trade_history.find({
  user_id: ObjectId("..."),
  closed_at: { $gte: new Date(Date.now() - 30*24*60*60*1000) }
}).sort({ closed_at: -1 }).hint("idx_user_exchange_closedat_desc");

// 2. Revenge Trading rule: trades opened in last 10 minutes
db.trade_history.find({
  user_id: ObjectId("..."),
  opened_at: { $gte: new Date(Date.now() - 10*60*1000) },
  leverage: { $gte: 20 }
}).sort({ opened_at: -1 });
```

2.4 Design Rationale

Compound Index: { user_id, exchange_id, closed_at: -1 } — The Primary Workhorse

This is the single most important index in the entire schema. The Quant Engine runs on a 60-second polling cycle and must efficiently fetch the last 30 days of trades for each user-exchange pair. The index structure follows the ESR rule (Equality-Sort-Range): user_id and exchange_id are equality filters (highest selectivity first), and closed_at is the sort/range key. This allows MongoDB to satisfy the query entirely from the index B-tree with a bounded range scan and no in-memory sort.

Deduplication Index on exchange_trade_id

The 60-second polling cycle will repeatedly fetch overlapping trade windows from exchange APIs. Without a deduplication guard, the collection would accumulate duplicate records, corrupting all aggregate calculations. The unique compound index on { user_id, exchange_id, exchange_trade_id } causes MongoDB to reject duplicate inserts with a DuplicateKey error, which the backend handles silently (upsert pattern). This is cheaper and more reliable than application-level deduplication.

Decimal128 for PnL Fields

> **Why Not Float64 (Number)?**
> Standard JavaScript/Python float64 (IEEE 754 double) cannot represent many decimal fractions exactly. For example: 0.1 + 0.2 == 0.30000000000000004 in float64. When accumulated across thousands of trades, these rounding errors corrupt PnL totals and Discipline Score components. MongoDB's Decimal128 type uses base-10 arithmetic with 34 significant digits — matching the precision of financial exchange data. All monetary fields (realized_pnl_usd, entry_price, etc.) must be written as NumberDecimal().

Denormalized is_win Boolean

The is_win field is a derived value (realized_pnl_usd > 0) that could be computed on the fly. It is deliberately stored as a denormalized field because the consecutive-loss-streak rule (BHV-005) requires a sorted scan filtered by is_win = false. Without the stored field, this would require a $expr operator on a Decimal128 comparison, which cannot use a standard index. The stored boolean allows a fast covered index scan.

raw_exchange_data Subdocument

The raw CCXT response is embedded for auditability and debugging. If a user disputes a calculated metric, engineers can inspect the original exchange payload without re-fetching. In production, this field can be moved to a cold-storage collection after 90 days if storage costs become a concern.

Sharding Strategy (Phase 2)

At scale (50M+ documents), the trade_history collection should be sharded on { user_id: 'hashed' }. Hashed sharding ensures even distribution of user data across shards and keeps all trades for a single user on the same shard (zone sharding optional for data residency). The primary workhorse index remains efficient within each shard.

3. Collection: risk_metrics

3.1 Purpose

The risk_metrics collection stores point-in-time snapshots of all computed Quant Engine outputs after every sync cycle. It serves two distinct read patterns: (1) fetching the single latest snapshot for dashboard rendering (low-latency point lookup), and (2) fetching a time-range of snapshots for drawing historical charts (Discipline Score trend, Max Drawdown history). It is also the source of truth for the SBT Oracle — the signed payload is derived from the most recent snapshot.

3.2 Schema Document

`© 2025 RiskHub. Confidential.

```
// risk_metrics — Document Schema (MongoDB JSON)
{
  _id: ObjectId("66b4c5d6e7f8g9h0i1j2k3l4"),

  // ── Partition Key ──────────────────────────────────────────
  user_id: ObjectId("64f1a2b3c4d5e6f7a8b9c0d1"), // INDEXED

  // ── Snapshot Timestamp (Time-Series Axis) ──────────────────
  calculated_at: ISODate("2024-03-10T14:05:00Z"), // Date, INDEXED (sort key)
  sync_cycle_id: "sync_20240310_140500_u7f1",     // String, unique per sync run

  // ── Calculation Window ─────────────────────────────────────
  window_days: 30,                      // Number, lookback window used
  trade_count: 47,                      // Number, trades included in this snapshot

  // ── Core Risk Metrics (SBT Payload Source of Truth) ────────
  max_drawdown: {
    value_pct: NumberDecimal("18.42"),  // Decimal128, peak-to-trough % drop
    value_usd: NumberDecimal("-412.30"), // Decimal128, absolute USD drawdown
    peak_equity_usd: NumberDecimal("2240.00"), // Decimal128
    trough_equity_usd: NumberDecimal("1827.70"), // Decimal128
    peak_at: ISODate("2024-02-20T10:00:00Z"),   // Date
    trough_at: ISODate("2024-03-01T15:30:00Z"), // Date
  },

  discipline_score: {
    total: 72,                          // Number (0-100), composite score,
INDEXED
    components: {
      leverage_consistency: 68,         // Number (0-100), component sub-score
      trade_frequency: 80,              // Number (0-100)
      post_loss_behavior: 55,           // Number (0-100)
      win_rate_consistency: 78,         // Number (0-100)
      drawdown_control: 71,             // Number (0-100)
    },
    grade: "C+",                        // String, derived label: A/B/C/D/F
    trend: "improving",                 // Enum: "improving" | "declining" |
"stable"
  },

  win_rate: {
    value_pct: NumberDecimal("57.45"),  // Decimal128, wins / total trades * 100
    wins: 27,                           // Number
    losses: 19,                         // Number
    breakeven: 1,                       // Number
  },

  leverage: {
    average: NumberDecimal("12.30"),    // Decimal128, mean leverage across all
trades
    median: NumberDecimal("10.00"),     // Decimal128
    max_used: 50,                       // Number, highest leverage in window
    std_dev: NumberDecimal("8.21"),     // Decimal128, consistency indicator
    over_20x_pct: NumberDecimal("14.89"), // Decimal128, % trades with leverage >
20x
  },

  // ── Additional Quant Metrics ───────────────────────────────
  profit_factor: NumberDecimal("1.42"), // Decimal128, gross profit / gross loss
  sharpe_ratio: NumberDecimal("0.87"),  // Decimal128, risk-adjusted return
  avg_trade_duration_seconds: 7200,    // Number, mean position hold time
  total_volume_usd: NumberDecimal("48320.00"), // Decimal128, cumulative notional
```

```
  net_pnl_usd: NumberDecimal("384.20"), // Decimal128, total realized PnL in
window
  net_pnl_pct: NumberDecimal("17.15"), // Decimal128, vs starting equity

  // ── Per-Exchange Breakdown (Denormalized for SBT detail) ──
  by_exchange: [
    {
      exchange_id: "binance",
      trade_count: 30,
      win_rate_pct: NumberDecimal("60.00"),
      avg_leverage: NumberDecimal("11.50"),
      net_pnl_usd: NumberDecimal("280.00"),
    },
    {
      exchange_id: "okx",
      trade_count: 17,
      win_rate_pct: NumberDecimal("52.94"),
      avg_leverage: NumberDecimal("13.80"),
      net_pnl_usd: NumberDecimal("104.20"),
    }
  ],

  // ── Active Rule Triggers (Snapshot of Firing Rules) ────────
  active_rule_flags: ["BHV-003"],      // Array<String>, rule IDs active at calc
time

  // ── SBT Payload (Pre-signed, ready for Oracle) ──────────────
  sbt_payload_hash: "0xkeccak256hash...", // String, keccak256 of the payload
  sbt_ready: true,                     // Boolean, sufficient data for mint

  // ── Metadata ────────────────────────────────────────────────
  schema_version: 1,
  calculation_duration_ms: 142         // Number, engine performance tracking
}
```

3.3 Index Definitions & Query Examples

```
// risk_metrics — createIndex() Statements + Query Examples
// ── Risk Metrics Collection Indexes ────────────────────────
// PRIMARY: Fetch latest snapshot for a user (dashboard load)
db.risk_metrics.createIndex(
  { user_id: 1, calculated_at: -1 },
  { name: "idx_user_calculated_desc" }
);

// CHART QUERIES: Time-series chart data (discipline score history)
// Covered index — no document fetch needed for charting
db.risk_metrics.createIndex(
  { user_id: 1, calculated_at: -1,
    "discipline_score.total": 1,
    "max_drawdown.value_pct": 1,
    "win_rate.value_pct": 1 },
  { name: "idx_user_chart_covered" }
);

// SBT LOOKUP: Find the snapshot used for a specific SBT mint
db.risk_metrics.createIndex(
  { user_id: 1, sbt_ready: 1, calculated_at: -1 },
  { name: "idx_sbt_ready_lookup" }
);
```

```
// TTL: Auto-delete snapshots older than 1 year (data hygiene)
// Keep only the last 365 days of history per user
db.risk_metrics.createIndex(
  { calculated_at: 1 },
  { expireAfterSeconds: 31536000, name: "idx_metrics_ttl_1year" }
);

// EXAMPLE QUERIES:
// 1. Dashboard: get latest snapshot
db.risk_metrics.findOne(
  { user_id: ObjectId("...") },
  { sort: { calculated_at: -1 } }
);

// 2. Chart API: discipline score over last 90 days
db.risk_metrics.find(
  { user_id: ObjectId("..."),
    calculated_at: { $gte: new Date(Date.now() - 90*24*60*60*1000) } },
  { projection: { calculated_at:1, "discipline_score.total":1,
      "max_drawdown.value_pct":1, "win_rate.value_pct":1 } }
).sort({ calculated_at: 1 });
```

3.4 Design Rationale

Append-Only Time-Series Log — No In-Place Updates

Every sync cycle creates a new document rather than updating an existing one. This is a fundamental time-series design principle: immutable append-only records. Benefits: (1) Full audit trail of how metrics evolved; (2) Chart data is a natural time-range query with no aggregation pipeline needed; (3) No write conflicts on the latest document from concurrent processes; (4) Rollback is trivial — just query a previous snapshot. The trade-off is storage growth, handled by the 1-year TTL index.

Covered Index for Chart Queries

Performance Pattern: Covered Index

The chart API endpoint needs to return (calculated_at, discipline_score.total, max_drawdown.value_pct, win_rate.value_pct) for up to 90 data points. The compound index idx_user_chart_covered includes all these fields. MongoDB can satisfy this query entirely from the index without loading the full documents from disk (a covered query). For a dashboard serving 1,000 concurrent users, this eliminates ~90% of disk I/O on the most frequent read endpoint.

by_exchange Denormalized Array

Per-exchange metric breakdowns are embedded as an array rather than as separate documents in a sub-collection. The dashboard per-exchange panel always reads all exchange breakdowns together (never one at a time), so embedding avoids N+1 query patterns. The array size is bounded by the number of connected exchanges (typically 2–5).

SBT Payload Preparation

The sbt_payload_hash field stores the keccak256 hash of the metrics payload at calculation time. When the user initiates SBT minting, the backend fetches the latest snapshot, verifies its hash (integrity check), signs it with the Oracle private key, and returns the signed payload to the frontend. Storing the

hash pre-computed eliminates a re-computation step in the critical mint path, reducing Oracle signing latency.

MongoDB Native Time-Series Collection (Future Optimization)

MongoDB 5.0+ offers a native Time Series Collection type with automatic bucket compression, optimized time-range queries, and reduced storage overhead for append-only numeric data. For Phase 2, risk_metrics should be migrated to a native time-series collection with timeField: 'calculated_at' and metaField: 'user_id'. This can reduce storage by 50–80% and improve range query performance further.

4. Collection: alerts_log

4.1 Purpose

The alerts_log collection stores every behavioral alert event generated by the Quant Engine. It must support three concurrent access patterns: (1) real-time alert delivery status tracking (was the WebSocket push successful?), (2) fast unread-count queries for the notification badge UI, and (3) paginated alert history with filtering by severity and rule type. The rate-limiting logic also queries this collection to prevent alert fatigue.

4.2 Schema Document

```
// alerts_log — Document Schema (MongoDB JSON)
{
  _id: ObjectId("67c5d6e7f8g9h0i1j2k3l4m5"),

  // ── Partition Key ──────────────────────────────────────────
  user_id: ObjectId("64f1a2b3c4d5e6f7a8b9c0d1"), // INDEXED

  // ── Rule Reference ─────────────────────────────────────────
  rule_id: "BHV-001",                  // String, INDEXED (maps to Quant Engine
rules)
  rule_name: "Revenge Trading",        // String, human-readable rule name
  rule_version: "1.0",                 // String, for rule logic versioning

  // ── Alert Classification ───────────────────────────────────
  severity: "warning",                 // Enum:
"critical"|"warning"|"caution"|"notice"
  category: "behavioral",             // Enum:
"behavioral"|"liquidation"|"portfolio"

  // ── Alert Content ──────────────────────────────────────────
  title: "Revenge Trading Detected",   // String, short display title
  message: "You opened a 20x BTCUSDT long position 6 minutes after a $72.50 loss.
Consider stepping away and reviewing your plan.", // String, rich contextual
message
  recommendation: "Step away for at least 30 minutes. Review your trade plan
before re-entering.", // String

  // ── Triggering Context (Quant Engine Evidence) ─────────────
  trigger_context: {
    rule_id: "BHV-001",
    loss_trade_id: ObjectId("65a3b4c5..."), // Ref -> trade_history._id
```

```
    loss_amount_usd: NumberDecimal("-72.50"),
    loss_pct_equity: NumberDecimal("-3.35"),
    trigger_trade_id: ObjectId("65a3b4c6..."), // Ref -> trade_history._id (new
position)
    trigger_leverage: 20,
    trigger_symbol: "BTCUSDT",
    minutes_since_loss: 6,              // Number, calculated trigger delta
    threshold_minutes: 10,              // Number, rule threshold for reference
    exchange_id: "binance",
  },

  // ── Related Trades (Array of references) ──────────────────
  related_trade_ids: [
    ObjectId("65a3b4c5d6e7f8g9h0i1j2k3"),  // loss trade
    ObjectId("65a3b4c6d6e7f8g9h0i1j2k4"),  // triggering trade
  ],

  // ── Delivery & Acknowledgement ────────────────────────────
  delivery: {
    websocket_sent: true,               // Boolean
    websocket_sent_at: ISODate("2024-03-10T09:16:02Z"), // Date
    email_sent: false,                  // Boolean
    email_sent_at: null,               // Date | null
  },

  is_read: false,                       // Boolean, INDEXED (for unread badge
count)
  read_at: null,                        // Date | null
  is_dismissed: false,                  // Boolean
  dismissed_at: null,                   // Date | null

  // ── Rate Limiting (Prevents alert fatigue) ────────────────
  rate_limit_key: "BHV-001::user_64f1a2b3::", // String, composite key
  rate_limit_window_start: ISODate("2024-03-10T09:00:00Z"), // Date, 30-min window
start

  // ── Timestamps ────────────────────────────────────────────
  triggered_at: ISODate("2024-03-10T09:16:00Z"),  // Date, INDEXED (rule fire
time)
  created_at: ISODate("2024-03-10T09:16:01Z"),    // Date (record creation, ~=
triggered_at)
  expires_at: ISODate("2024-04-09T09:16:01Z"),    // Date, TTL: auto-delete after
30 days

  // ── Metadata ──────────────────────────────────────────────
  schema_version: 1
}
```

4.3 Index Definitions & Query Examples

```
// alerts_log — createIndex() Statements + Query Examples
// ── Alerts Log Collection Indexes ──────────────────────────
// PRIMARY: Alert history page (recent alerts, paginated)
db.alerts_log.createIndex(
  { user_id: 1, triggered_at: -1 },
  { name: "idx_user_triggered_desc" }
);

// UNREAD BADGE: Fast count of unread alerts for UI badge
db.alerts_log.createIndex(
```

```
  { user_id: 1, is_read: 1, triggered_at: -1 },
  { name: "idx_user_isread_triggered" }
);

// RULE FILTER: "Show me all Revenge Trading alerts"
db.alerts_log.createIndex(
  { user_id: 1, rule_id: 1, triggered_at: -1 },
  { name: "idx_user_ruleid_triggered" }
);

// SEVERITY FILTER: Critical alerts dashboard panel
db.alerts_log.createIndex(
  { user_id: 1, severity: 1, triggered_at: -1 },
  { name: "idx_user_severity_triggered" }
);

// RATE LIMITING: Check if rule already fired in 30-min window
db.alerts_log.createIndex(
  { rate_limit_key: 1, triggered_at: -1 },
  { name: "idx_rate_limit_key" }
);

// TTL: Auto-delete alerts older than 30 days
db.alerts_log.createIndex(
  { expires_at: 1 },
  { expireAfterSeconds: 0, name: "idx_alerts_ttl_30days" }
  // expires_at is set to triggered_at + 30 days at write time
);

// EXAMPLE QUERIES:
// 1. Unread alert count for notification badge
db.alerts_log.countDocuments({
  user_id: ObjectId("..."),
  is_read: false
});

// 2. Rate limit check before firing a new alert
db.alerts_log.findOne({
  rate_limit_key: "BHV-001::user_64f1a2b3::",
  triggered_at: { $gte: new Date(Date.now() - 30*60*1000) }
});

// 3. Mark all as read (bulk update)
db.alerts_log.updateMany(
  { user_id: ObjectId("..."), is_read: false },
  { $set: { is_read: true, read_at: new Date() } }
);
```

4.4 Design Rationale

trigger_context Subdocument — Embedded Evidence

The trigger_context object embeds a snapshot of the exact data that fired the rule: the loss amount, the triggering trade's leverage, the time delta, and the threshold values. This is critical for two reasons: (1) the alert message can be rendered with precise, user-specific data (e.g., 'You opened a 20x position 6 minutes after a $72.50 loss') rather than a generic string; (2) if the trade_history records are later modified or purged, the alert evidence remains intact for audit purposes.

Rate Limiting via rate_limit_key Index

Alert Fatigue Prevention Design

The Quant Engine evaluates rules on every 60-second sync cycle. Without rate limiting, a persistent revenge-trading pattern could generate 60 alerts per hour. The rate_limit_key field is a composite string (rule_id + user_id) indexed for fast point lookup. Before firing any alert, the engine queries: findOne({ rate_limit_key: '...', triggered_at: { $gte: 30-min-ago } }). If a document exists, the alert is suppressed. This check adds ~1ms latency per rule evaluation and prevents alert storms.

related_trade_ids as ObjectId Array

The alert stores references (ObjectId) to the specific trades in trade_history that caused the rule to fire. This enables the frontend alert detail view to deep-link directly to the relevant trades, giving users actionable context. Array references are preferred over embedding full trade documents because trade records can be updated (e.g., fee reconciliation), and embedded copies would become stale.

expires_at TTL vs Static TTL Index

Rather than using a static expireAfterSeconds on created_at (which would delete all documents older than N seconds from creation), the schema uses an explicit expires_at field with expireAfterSeconds: 0. This means MongoDB deletes the document when the current time passes expires_at. This approach allows future flexibility: high-severity alerts (critical liquidation warnings) could be given a 90-day retention period by writing a farther-future expires_at value, while routine notices expire in 30 days — all served by the same index.

5. Complete Index Summary

The table below lists all indexes across all four collections for operational reference (DevOps, monitoring, and index management).

Collection	Index Name	Fields	Type	Purpose
users	idx_email_unique	{ email: 1 }	Unique	Login lookup
users	idx_wallet_address_unique	{ wallet.address: 1 }	Unique + Sparse	SIWE / SBT lookup
users	idx_exchange_keys_exchange_id	{ exchange_keys.exchange_id: 1 }	Standard	Ops monitoring
users	idx_sbt_status_tokenid	{ sbt.status: 1, sbt.token_id: 1 }	Sparse	SBT job queue
users	idx_siwe_nonce_ttl	{ wallet.siwe_nonce_expires_at: 1 }	TTL	Auto-expire nonces
trade_history	idx_user_exchange_closedat_desc	{ user_id:1, exchange_id:1, closed_at:-1 }	Compound	PRIMARY workhorse
trade_history	idx_dedup_trade	{ user_id:1, exchange_id:1, exchange_trade_id:1 }	Unique	Deduplication

`© 2025 RiskHub. Confidential.

Collection	Index Name	Fields	Type	Purpose
trade_history	idx_user_symbol_closedat	{ user_id:1, symbol:1, closed_at:-1 }	Compound	Contagion graph
trade_history	idx_user_openedat_desc	{ user_id:1, opened_at:-1 }	Compound	Revenge trading rule
trade_history	idx_user_iswin_closedat	{ user_id:1, is_win:1, closed_at:-1 }	Compound	Loss streak rule
risk_metrics	idx_user_calculated_desc	{ user_id:1, calculated_at:-1 }	Compound	Dashboard latest
risk_metrics	idx_user_chart_covered	{ user_id:1, calculated_at:-1, score, drawdown, winrate }	Covered	Chart API
risk_metrics	idx_sbt_ready_lookup	{ user_id:1, sbt_ready:1, calculated_at:-1 }	Compound	SBT mint flow
risk_metrics	idx_metrics_ttl_1year	{ calculated_at: 1 }	TTL	1-year auto-purge
alerts_log	idx_user_triggered_desc	{ user_id:1, triggered_at:-1 }	Compound	Alert history page
alerts_log	idx_user_isread_triggered	{ user_id:1, is_read:1, triggered_at:-1 }	Compound	Unread badge count
alerts_log	idx_user_ruleid_triggered	{ user_id:1, rule_id:1, triggered_at:-1 }	Compound	Rule filter view
alerts_log	idx_user_severity_triggered	{ user_id:1, severity:1, triggered_at:-1 }	Compound	Severity filter
alerts_log	idx_rate_limit_key	{ rate_limit_key:1, triggered_at:-1 }	Compound	Rate limiter
alerts_log	idx_alerts_ttl_30days	{ expires_at: 1 }	TTL	30-day auto-purge

6. Query Performance Guidelines for Backend Engineers

6.1 Always Use hint() in Quant Engine Queries

The Quant Engine runs on a tight 60-second cycle. MongoDB's query planner can sometimes choose a suboptimal index on complex queries. Explicitly hint the index name on all Quant Engine queries (as shown in the code blocks above) to guarantee plan stability and prevent unexpected performance degradation after collection statistics change.

6.2 Use Projection to Minimize Document Transfer

When building the dashboard, always specify a projection object to return only required fields. For example, the risk summary widget needs only discipline_score.total, max_drawdown.value_pct,

`© 2025 RiskHub. Confidential.

win_rate.value_pct, and calculated_at — not the full 40-field document. Projection reduces network transfer, deserialisation time, and application memory usage.

6.3 Prefer Upsert over findOne + insert for Trade Ingestion

The trade ingestion loop should use bulkWrite() with updateOne + upsert:true operations, keyed on the deduplication index fields. This is a single round-trip to MongoDB regardless of batch size, compared to N individual insert calls. Recommended batch size: 500 documents per bulkWrite call. The deduplication unique index guarantees idempotency automatically.

6.4 Connection Pooling

The FastAPI backend uses Motor (async MongoDB driver). Configure the connection pool with minPoolSize: 5, maxPoolSize: 50 for the MVP. The Quant Engine's parallel exchange polling creates bursts of concurrent database writes — the pool prevents connection exhaustion. Monitor the pool utilization metric in MongoDB Atlas and increase maxPoolSize if avg wait time exceeds 10ms.

END OF DOCUMENT — RiskHub MongoDB Schema v1.0

`© 2025 RiskHub. Confidential.